

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Support for Neural Network Activation Functions in TAFFO

Project for "Code Optimization and Transformation" course
Extension of Value Range Analysis and Fixed-Point Conversion Using Lookup Tables

Francesco Egidio Faggion, 10742119

Advisor:
Prof. Agosta Giovanni

Co-advisors:
Niccolosi Niccolò
Magnani Gabriele

Academic year:
2025-2026

Abstract: This work extends TAFFO with support for four neural-network activation functions: ReLU, Sigmoid, hyperbolic tangent, and Swish. Dedicated Value Range Analysis models determine their output ranges, including the internal minimum of the non-monotonic Swish function. During fixed-point Conversion, ReLU is lowered directly to integer operations, whereas the other activations are replaced by 1024-entry lookup tables constructed according to the formats selected by Data Type Allocation. The implementation supports TAFFO-generated function clones and was validated with LLVM 15 and LLVM 18. End-to-end tests confirmed the correctness of the transformed programs, while all 3072 generated lookup-table entries matched independently quantized reference values.

1. Introduction and Objectives

Activation functions are a key element of neural networks because they introduce non-linearities into the model, enabling it to represent a much wider range of functions. Without non-linear activation functions, a sequence of neural-network layers would still behave as a single affine transformation. Commonly used activation functions include ReLU, tanh, and sigmoid. In addition to these three functions, this project also considers Swish, defined as x sigmoid(x), because it provides a relevant non-monotonic case.
TAFFO is an LLVM-based precision-tuning framework that replaces floating-point computations with reduced-precision representations, particularly fixed point [1]. This optimization can improve performance and reduce computational costs on architectures where fixed-point arithmetic is more efficient than floating-point arithmetic. The objective of this project is to enable TAFFO to recognize these activation functions and process them throughout its precision-tuning pipeline. In particular, calls to tanh, sigmoid, and Swish are replaced with accesses to precomputed lookup tables (LUTs) whose entries contain fixed-point results.
To achieve this objective, two main extensions are required:

1. extending Value Range Analysis (VRA) so that TAFFO can compute the output ranges of the supported activation functions;
2. implementing their fixed-point conversion, using LUTs for tanh, sigmoid, and Swish and direct integer operations for ReLU.

ReLU does not require a lookup table because its result can be computed directly using efficient integer operations. The implementation details are discussed in section 3.

2. Background

2.1. Floating-point vs Fixed-point comparison

While floating-point representation uses a sign, an exponent, and a significand, a fixed-point representation stores a raw integer, interpreted as either signed or unsigned, together with an implicit scaling factor. If F denotes the number of fractional bits, the represented value is

$$x = x_{\text{raw}}\, 2^{-F}. \tag{1}$$

Floating-point numbers have non-uniform absolute spacing: the distance between adjacent representable values increases with their magnitude. Fixed-point formats instead represent uniformly spaced values, with 2^{-F} being the distance between two consecutive representable numbers. A fixed-point format must therefore balance range and precision, making the choice of total bit width, signedness, and number of fractional bits crucial.

2.2. TAFFO Pipeline

TAFFO uses source-code annotations to identify the values involved in precision tuning and to receive information that cannot always be inferred at compile time. In particular, the programmer should provide the ranges of input variables whose values depend on external data. For example:

```
float x __attribute__((annotate(
    "target('main') scalar(range(-2, 1))"
)));
```

The annotation contains the following elements:

- `target('main')` marks the function containing the declaration as a starting point for the analyses; the supplied string is also used as a label by TAFFO's error-reporting infrastructure;
- `scalar(...)` specifies that the annotated variable contains homogeneous data;
- `range(-2, 1)` states that the runtime value of `x` belongs to the interval $[-2, 1]$.

Starting from these annotations, TAFFO processes the program through four main stages. The Initializer parses the annotations and identifies the values involved in the transformation. VRA propagates the known intervals through LLVM IR. Data Type Allocation (DTA) selects an appropriate representation for each value, including signedness, total bit width, and fractional bit width. Finally, Conversion rewrites the LLVM IR using the selected types. The annotated program can be compiled by replacing a standard compiler invocation with, for example:

```
taffo -O3 program.c -o program-taffo
```

2.3. Activation Functions

The activation functions selected for this project are summarized in table 1. Their mathematical properties determine both how VRA computes their output intervals and how they are converted to fixed-point operations.

Table 1: Activation functions supported by the project.

Function	**Definition**	**Relevant property**
ReLU	$\max(0, x)$	Monotonically increasing and piecewise linear.
Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$	Monotonically increasing, with output in $(0, 1)$.
tanh	$\tanh(x)$	Monotonically increasing, with output in $(-1, 1)$.
Swish	$x\,\sigma(x)$	Non-monotonic and characterized by an internal minimum.

3. Implementation

3.1. Activation Recognition and Clone Handling

Calls to supported activation functions are recognized from their function names. The accepted names include the standard variants associated with different floating-point types, such as `sigmoid`, `sigmoidf`, and `sigmoidl`. The same mechanism is used for ReLU, tanh, and Swish.
TAFFO may clone functions while processing the program. Consequently, the recognition mechanism also accepts names such as:

```
sigmoid_clone0
tanhf_clone2
swishl_clone17
```

The activation family is selected with checks such as:

```
if (isExactFunctionOrClone(name, "sigmoid") ||
    isExactFunctionOrClone(name, "sigmoidf") ||
    isExactFunctionOrClone(name, "sigmoidl"))
  return SIGMOID;
```

The `isExactFunctionOrClone` helper accepts either the exact base name or the base name followed by the suffix `_clone` and one or more decimal digits. This restriction avoids incorrectly recognizing unrelated functions whose names merely begin with the same prefix. For example, `sigmoid_clone3` is accepted, whereas `sigmoid_approx` is not.
This helper is used during Conversion. VRA performs an analogous normalization through `getOriginalFunctionName`, which removes a valid `_cloneN` suffix before looking up the original function in the range-analysis whitelist. Thus, both stages recognize TAFFO-generated clones while rejecting unrelated functions with similar prefixes.
Function recognition introduces an implicit semantic contract: a function recognized as `sigmoid`, for example, is assumed to implement the mathematical sigmoid. TAFFO does not inspect its body to verify this property. Instead, VRA applies the corresponding mathematical range model, and Conversion replaces the call with the supported fixed-point implementation.

3.2. Range Computation

Let the input range of an activation function be the interval $[a, b]$. For a monotonically increasing function f, interval propagation is performed by evaluating the two endpoints:

$$f([a, b]) = [f(a), f(b)]. \tag{2}$$

This rule applies directly to tanh and sigmoid:

$$\tanh([a, b]) = [\tanh(a), \tanh(b)], \tag{3}$$
$$\sigma([a, b]) = [\sigma(a), \sigma(b)], \tag{4}$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \tag{5}$$

ReLU is also monotonically increasing, but its piecewise-linear definition allows its output interval to be written directly as:

$$\mathrm{ReLU}([a, b]) = [\max(0, a), \max(0, b)]. \tag{6}$$

Swish requires different handling because it is not monotonic over its entire domain. It is defined as

$$s(x) = x\sigma(x) \tag{7}$$

and has a unique finite minimum at approximately

$$c = -1.2784645427610737. \tag{8}$$

For an input interval $[a, b]$, the set of candidate values is therefore

$$S = \{s(a), s(b)\} \cup \begin{cases} \{s(c)\}, & \text{if } a \le c \le b, \\ \emptyset, & \text{otherwise.} \end{cases} \tag{9}$$

The resulting output interval is

$$s([a, b]) = [\min S, \max S]. \tag{10}$$

Therefore, evaluating only the endpoints would be insufficient whenever the input interval contains the internal minimum. The implementation explicitly checks whether c belongs to the input interval and includes $s(c)$ among the candidates when necessary.

3.3. Fixed-point Conversion

ReLU is converted directly into integer LLVM IR operations, without requiring a lookup table. The Conversion pass first retrieves the fixed-point type selected by DTA and converts the input operand accordingly. For a signed representation, it generates a signed comparison with zero followed by a select instruction, implementing $\max(0, x)$. For an unsigned representation, the input cannot be negative, so ReLU is equivalent to the identity function. In this case, an explicit identity instruction is generated to preserve TAFFO's type information, even when the input is an LLVM constant.

Tanh, sigmoid, and Swish are converted using a common LUT-based infrastructure. The only function-specific component is the mathematical evaluator used to generate the table entries:

```
Value* ConversionPass::createTanh(CallBase* call) {
  return createLUTActivation(call, "tanh", evaluateTanh);
}
```

and similarly for Sigmoid and Swish.

The generic `createLUTActivation` function first retrieves the input and output fixed-point types selected by DTA. These two types may differ in signedness, total bit width, and number of fractional bits. The input type determines the domain sampled by the table and the runtime indexing procedure, whereas the output type determines how every activation result is quantized and stored.
For each combination of activation function, input type, and output type, TAFFO generates a constant table containing $2^{10} = 1024$ elements. During table generation, each 10-bit index is interpreted as a fixed-point bit pattern. For signed inputs, indices from 0 to 511 represent non-negative values, while indices from 512 to 1023 represent negative two's-complement values. For unsigned inputs, all indices represent non-negative values. The corresponding real input is reconstructed by applying the scaling factor of the reduced 10-bit format, after which the selected activation function is evaluated:

$$x_i = r_i \, 2^{-F_{\mathrm{LUT}}}, \qquad y_i = f(x_i), \tag{11}$$

where r_i is the signed or unsigned raw value represented by index i, F_{LUT} is the effective number of fractional bits after input reduction, and f is tanh, sigmoid, or Swish. The resulting value y_i is then quantized according to the output fixed-point type, rounded toward negative infinity, and saturated when it exceeds the representable output range.
At runtime, the input is reduced to a 10-bit pattern by retaining its ten most significant bits. An arithmetic right shift is used for signed inputs to preserve the sign, whereas a logical right shift is used for unsigned inputs. The resulting pattern is interpreted as an unsigned table index:

```
shift = inputBits - 10;
reduced = inputIsSigned
              ? arithmeticRightShift(input, shift)
              : logicalRightShift(input, shift);
index = zeroExtend(truncateTo10Bits(reduced));
result = activationLUT[index];
```

Consequently, the original floating-point function call is replaced with integer shift, truncation, extension, address-computation, and load instructions. The lookup covers the entire domain representable by the fixed-point input type, rather than only the exact interval computed by VRA. Inputs that share the same ten most

Activation	Input range	Computed output range
ReLU (signed)	$[-2, 3]$	$[0, 3]$
ReLU (unsigned)	$[0, 3]$	$[0, 3]$
Sigmoid	$[-2, 1]$	$[0.119203, 0.731059]$
tanh	$[-2, 1]$	$[-0.964028, 0.761594]$
Swish	$[-2, 1]$	$[-0.278465, 0.731059]$

Table 2: Ranges inferred by TAFFO's VRA.

significant bits therefore access the same table entry, trading accuracy for a fixed table size and an efficient runtime implementation.
The generated table is stored as an internal constant LLVM global. Its name encodes the activation function and the input and output types, allowing the same table to be reused when multiple calls have an identical configuration. Sigmoid is evaluated through separate expressions for negative and non-negative inputs to avoid the risk of exponential overflow, while tanh and Swish use their corresponding mathematical definitions.

4. Experimental Validation

The implementation was validated at three different levels: correctness of the ranges computed by VRA, structure of the generated LLVM IR, and numerical accuracy of the resulting fixed-point programs. The regression tests were executed with LLVM 15 on macOS and with LLVM 18.1.8 in a Linux Docker environment. The LLVM 18 build required two minor API-compatibility changes in code inherited from the master branch; these changes were unrelated to the activation-function implementation.

Value Range Analysis. The VRA results were inspected in the intermediate TAFFO metadata. Table 2 compares the input ranges provided through annotations with the ranges inferred for the activation calls.
The Swish result is particularly relevant because the function is not monotonic over the tested interval. Its minimum is attained at the internal stationary point $x \simeq -1.27846$, rather than at one of the interval endpoints. The inferred minimum, approximately -0.278465, therefore confirms that the dedicated non-monotonic range model is used correctly. The metadata also contained calls such as `relu_clone0`, `sigmoid_clone0`, and `swish_clone0`, confirming that the clone-aware function recognition works as intended.

Generated LLVM IR. Inspection of the final IR confirmed that no calls to the original floating-point activation functions remained after Conversion. Signed ReLU was lowered to a signed integer comparison followed by a `select`, while unsigned ReLU was represented by an explicit identity operation, since its input cannot be negative.
For Sigmoid, tanh, and Swish, the generated code first shifts the fixed-point input, truncates it to an `i10` bit pattern, zero-extends the pattern to obtain an index between 0 and 1023, and finally loads the corresponding value from a `[1024 x i32]` table. The selected fixed-point formats were also visible in the generated symbols. For example, Sigmoid converted a signed input with 29 fractional bits into an unsigned output with 31 fractional bits, whereas tanh and Swish used signed outputs with 29 fractional bits.

Lookup-table validation. All 1024 entries of each LUT were compared with independently computed reference values. For every sampling point x_i, the expected raw fixed-point value was calculated as

$$q_i = \mathrm{sat}\left(\left\lfloor f(x_i)\, 2^F \right\rfloor\right),$$

where F is the number of fractional bits and sat restricts the result to the representable output range. No mismatch was found among the 3072 entries. The maximum output-quantization errors at the LUT sampling points are reported in Table 3. These values measure the quantization of the stored outputs and do not include the additional approximation error that may arise for arbitrary inputs mapped to the same LUT entry.

End-to-end tests. Finally, each test program was compiled both normally and through the complete TAFFO pipeline. The resulting executables were run on representative values at the endpoints of the annotated ranges, and their outputs were compared. ReLU produced an exact result, while the maximum absolute errors were approximately 1.91×10^{-8} for Sigmoid, 2.05×10^{-8} for tanh, and 2.05×10^{-8} for Swish. These tests verify the complete path from VRA and DTA to Conversion, LUT generation, and execution of the transformed program.

Activation	Mismatching entries	Quantization error
Sigmoid	0/1024	4.656×10^{-10}
tanh	0/1024	1.861×10^{-9}
Swish	0/1024	1.862×10^{-9}

Table 3: Validation of output quantization at the LUT sampling points.

The tests focus on functional correctness rather than performance. Consequently, no speedup is reported for these small regression programs.

Limitation encountered during testing. An additional dense runtime sweep was initially considered to evaluate the activation functions over a larger number of input values. In this test, the floating-point input was generated from an integer index. TAFFO converted the index to fixed point through a left shift performed within the original 32-bit integer width, causing overflow for sufficiently large index values and consequently producing incorrect, periodic inputs. The same behaviour was reproduced with both LLVM 15 and LLVM 18. Since the problem occurs during the general integer-to-fixed-point conversion, before the activation function is evaluated, it is not caused by the activation-function extension. The dense sweep was therefore replaced by the direct validation of all 1024 generated LUT entries.

5. Conclusions and Future Work

This project extended the TAFFO precision-tuning pipeline with explicit support for ReLU, Sigmoid, hyperbolic tangent, and Swish. The work covers both the analysis and conversion stages. VRA can now propagate the output ranges of these activation functions, including the non-monotonic behaviour of Swish, while Conversion can replace their floating-point calls with fixed-point implementations. ReLU is lowered to a direct integer clamp, with separate handling for signed and unsigned inputs, whereas Sigmoid, tanh, and Swish reuse a common lookup-table infrastructure. Function recognition also accounts for clones generated during the TAFFO pipeline.
The implementation was validated through end-to-end regression tests, inspection of the generated LLVM IR, and independent verification of every generated LUT entry. The tests confirmed that the original activation calls are eliminated, the formats selected by DTA are respected, and the resulting fixed-point programs closely reproduce their floating-point counterparts. The same functionality was tested with LLVM 15 and LLVM 18.
Future work could extend the same infrastructure to additional activation functions, such as Leaky ReLU, Softplus, or GELU. The fixed LUT size could also be made configurable to investigate the trade-off between memory consumption, lookup precision, and execution time. Polynomial or piecewise approximations could be compared with LUT-based conversion, particularly on embedded targets where memory is limited. Finally, evaluation on complete neural-network workloads would make it possible to measure the effects on performance, memory usage, and overall inference accuracy. The general integer-to-fixed-point conversion issue encountered while constructing a dense runtime sweep could be investigated separately, since it is independent of the activation-function support introduced in this work.

References

[1] Daniele Cattaneo, Michele Chiari, Giovanni Agosta, and Stefano Cherubin. Taffo: The compiler-based precision tuner. *SoftwareX*, 20:101238, 2022.

[2] Stefano Cherubin, Daniele Cattaneo, Michele Chiari, Antonio Di Bello, and Giovanni Agosta. Taffo: Tuning assistant for floating to fixed point optimization. *IEEE Embedded Systems Letters*, 12(1):5–8, 2019.

[3] Gabriele Magnani, Daniele Cattaneo, Lev Denisov, Giuseppe Tagliavini, Giovanni Agosta, and Stefano Cherubin. Synergistic memory optimisations: precision tuning in heterogeneous memory hierarchies. *IEEE Transactions on Computers*, 74(9):3168–3180, 2025.